DARSHAN UNIVERSITY

Department of Computer Science & Engineering

Course Code: 2601CS102

Web Designing

B.Tech. First Semester

CHAPTER 2

CSS — Cascading Style Sheets

E-Notes / Study Material

Topic	Coverage
Introduction to CSS	What CSS is, why it matters, how it works with HTML
Syntax & Structure	Selectors, properties, values, declaration blocks
Types of CSS	Inline, Internal, External CSS
Targeting using CSS	Element, class, ID, group, descendant & pseudo selectors
Background Properties	Colors, images, repeat, size, position, gradients
Text & Font	Typography properties, Google Fonts, fallback fonts
The Box Model	Margin, border, padding, content, box-sizing
Styling Lists	list-style properties, custom bullets, nav menus
CSS Positioning	static, relative, absolute, fixed, sticky, z-index

Academic Year 2025–26

1. Introduction to CSS

1.1 What is CSS?

CSS stands for Cascading Style Sheets. It is a stylesheet language used to describe the visual presentation of an HTML document — colors, fonts, spacing, layout, and animations. While HTML defines the structure and content of a webpage, CSS defines how that content looks.

> **Key Facts About CSS:**
>
> Created by Håkon Wium Lie in 1994, officially released as a W3C standard in 1996
>
> Current version in active use: CSS3, developed as separate modules (Selectors, Backgrounds, Flexbox, Grid, Animations, etc.)
>
> "Cascading" refers to how styles flow down and combine from multiple sources — browser defaults, external files, internal styles, and inline styles
>
> CSS can style not just HTML but also XML-based documents like SVG and XHTML

1.2 Why Do We Need CSS?

Without CSS	With CSS
Plain black text on white background	Full control over colors, fonts, and layout
No visual hierarchy or branding	Consistent design across the entire website
Every page looks identical & boring	Unique, professional, and engaging pages
Styling mixed inside HTML (messy)	Content and design kept separate (clean code)
Hard to maintain — repeat changes everywhere	Change one CSS file → updates the whole site

> 💡 **Real-World Analogy:**
>
> HTML is like the walls and rooms of a house (structure).
>
> CSS is the paint, furniture, and interior design (presentation).
>
> The same HTML "house" can look completely different with different CSS "decoration".

2. Basic Syntax and Structure of CSS

2.1 CSS Rule Anatomy

Every CSS rule follows the same fundamental structure — a selector followed by a declaration block containing one or more property-value pairs.

```
selector {
  property: value;
  property: value;
}

/* Real Example */
p {
  color: navy;
  font-size: 16px;
  line-height: 1.5;
}
```

Part	Meaning	Example
Selector	Targets which HTML element(s) to style	p, .box, #header
Declaration Block	Everything inside the curly braces { }	{ color: red; }
Property	The style aspect being changed	color, font-size, margin
Value	The setting applied to that property	red, 16px, 10px auto
Declaration	One property : value pair	color: red;

> ⚠️ **Syntax Rules to Remember:**
> Every declaration MUST end with a semicolon (;) — forgetting it can break the rest of the block
> Property and value are separated by a colon (:), never an equals sign
> CSS is NOT case-sensitive for properties/values, but class & ID names ARE case-sensitive
> Whitespace, indentation, and line breaks are ignored by the browser — use them for readability

2.2 CSS Comments

Comments are notes for developers that the browser completely ignores. They help explain sections of your stylesheet:

```
/* This is a single-line comment */

/*
   This is a
   multi-line comment
*/

h1 {
  color: teal; /* Main heading color */
}
```

3. Types of CSS

There are three ways to apply CSS to an HTML document. Each has different use cases and levels of priority.

3.1 Inline CSS

Written directly inside an HTML tag using the style attribute. Applies only to that single element.

```
<p style="color: red; font-size: 18px;">This is inline styled text.</p>
```

3.2 Internal CSS

Written inside a <style> tag within the <head> section. Applies to the entire single page it's written on.

```
<head>
  <style>
    p {
      color: navy;
      font-size: 16px;
    }
    h1 {
      text-align: center;
    }
  </style>
</head>
```

3.3 External CSS

Written in a completely separate .css file and linked to the HTML document using the <link> tag. Applies to every page that links to it.

```
<head>
  <link rel="stylesheet" href="style.css">
</head>

/* style.css file content */
p {
  color: navy;
  font-size: 16px;
}
```

3.4 Comparison Table

Feature	Inline	Internal	External
Location	Inside HTML tag	Inside <style> in <head>	Separate .css file
Scope	One element only	One page only	Unlimited pages
Maintainability	Poor — hard to manage	Moderate	Best — single source
Load Speed	No extra request	No extra request	Cached after first load

Best Use Case	Quick single-time test	Small single-page site	Real-world projects

> 📌 **Priority Order (Specificity/Cascade):**
> Inline CSS > Internal CSS > External CSS > Browser Default Styles
> When conflicting styles exist, the one with higher priority (closer to "inline") wins.
> Best Practice: Always use External CSS for real projects — keeps HTML clean and styles reusable.

4. Targeting Using CSS (Selectors)

Selectors determine exactly which HTML elements a CSS rule applies to. Mastering selectors — from the basics through combinators, attribute matching, and pseudo-classes/elements — is essential to writing precise, maintainable stylesheets.

4.1 Basic Selectors

```
/* Element Selector — targets ALL <p> tags */
p { color: black; }

/* Class Selector — targets elements with class="box" */
.box { border: 1px solid gray; }

/* ID Selector — targets the ONE element with id="header" */
#header { background-color: navy; }

/* Universal Selector — targets EVERY element on the page */
* { margin: 0; padding: 0; }

/* Grouping Selector — apply same rule to multiple selectors */
h1, h2, h3 { font-family: Arial, sans-serif; }
```

Selector	Syntax	Targets
Element	p { }	Every <p> element
Class	.classname { }	Every element with that class (reusable)
ID	#idname { }	The single element with that ID (unique)
Universal	* { }	Absolutely every element
Grouped	h1, p { }	All listed selectors, same rule

4.2 Advanced Selectors — Combinators

Combinators let you target elements based on their relationship to other elements in the HTML tree, rather than styling every matching element globally.

```
/* Descendant Combinator (space) — ANY <p> inside a <div>, at any depth */
div p { color: gray; }

/* Child Combinator (>) — ONLY a <p> that is a DIRECT child of <div> */
div > p { color: blue; }

/* Adjacent Sibling Combinator (+) — <p> immediately after an <h1> */
h1 + p { font-weight: bold; }

/* General Sibling Combinator (~) — ALL <p> elements after an <h1>, same parent */
h1 ~ p { color: green; }
```

Combinator	Symbol	Matches
Descendant	(space)	Any depth inside the ancestor — grandchildren included
Child	>	Only DIRECT children, one level down
Adjacent Sibling	+	The single element immediately following, same parent
General Sibling	~	All following elements, same parent

> 💡 **Descendant vs Child — the #1 confusion:**
> div p → matches a <p> ANYWHERE inside the div (even nested 3 levels deep)
> div > p → matches a <p> ONLY if it is an immediate child of the div
> Rule of thumb: use > when you want to avoid accidentally styling nested/unrelated elements.

4.3 Advanced Selectors — Attribute Selectors

Attribute selectors match elements based on the presence or value of an HTML attribute — extremely useful for styling forms and links without adding extra classes.

```
/* [attr] — element simply HAS the attribute */
input[required] { border-color: red; }

/* [attr="value"] — attribute equals EXACTLY this value */
input[type="text"] { border: 1px solid teal; }

/* [attr^="value"] — attribute STARTS WITH this value */
a[href^="https"] { color: green; }        /* secure links */

/* [attr$="value"] — attribute ENDS WITH this value */
a[href$=".pdf"] { color: red; }           /* links to PDF files */

/* [attr*="value"] — attribute CONTAINS this value anywhere */
a[href*="youtube"] { color: darkred; }    /* any YouTube link */

/* [attr~="value"] — attribute contains this word in a space-separated list */
img[class~="thumbnail"] { border-radius: 4px; }
```

Syntax	Meaning	Example Use Case
[attr]	Has the attribute at all	Highlight all required form fields
[attr="val"]	Exact match	Style only type="submit" buttons
[attr^="val"]	Starts with	Style all external (https) links
[attr$="val"]	Ends with	Add a PDF icon to all .pdf links
[attr*="val"]	Contains anywhere	Style links containing "promo"

4.4 Pseudo-Classes — Styling Based on State

A pseudo-class (single colon :) targets an element based on a special STATE or POSITION — something that isn't directly written in the HTML, such as being hovered, focused, or being the 3rd child of its parent.

▸ Interaction (User Action) Pseudo-Classes

Pseudo-Class	Triggers When
:hover	Mouse pointer is resting over the element
:focus	Element is clicked into or reached via Tab key (inputs, links)
:active	Element is being actively clicked (mouse button down)
:visited	A link the user has already clicked before
:link	A link the user has NOT yet visited
:focus-within	Applied to a parent when ANY child inside it has focus
:disabled / :enabled	Form field is disabled or enabled
:checked	A checkbox or radio button that is currently checked

```
a:link    { color: blue; }     /* unvisited link */
a:visited { color: purple; }   /* already-visited link */
a:hover   { color: red; text-decoration: underline; }
a:active  { color: orange; }

input:focus { border-color: teal; outline: none; box-shadow: 0 0 4px teal; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
input[type="checkbox"]:checked + label { font-weight: bold; }
```

▸ Structural (Position-in-Parent) Pseudo-Classes

Pseudo-Class	Matches
:first-child	Element that is the FIRST child of its parent
:last-child	Element that is the LAST child of its parent
:only-child	Element that is the ONLY child of its parent
:nth-child(n)	The nth child — n can be a number, even, odd, or formula (3n)
:nth-last-child(n)	The nth child, counted from the END
:first-of-type / :last-of-type	First/last element of its specific tag type
:not(selector)	Matches everything EXCEPT what's inside the parentheses

```
li:first-child { font-weight: bold; }
li:last-child  { border-bottom: none; }

/* Zebra-striping a table without extra classes */
tr:nth-child(odd)  { background: #f4f4f4; }
tr:nth-child(even) { background: #ffffff; }
```

```
/* Every 3rd card in a grid gets extra margin */
.card:nth-child(3n) { margin-right: 0; }

/* Style every button EXCEPT the one with class .btn-danger */
button:not(.btn-danger) { background: teal; }
```

> 💡 **nth-child() Formula Cheat Sheet:**
> nth-child(odd) → 1st, 3rd, 5th... (great for striped tables)
> nth-child(even) → 2nd, 4th, 6th...
> nth-child(3n) → every 3rd element (3rd, 6th, 9th...)
> nth-child(3n+1) → 1st, 4th, 7th... (offset pattern)

4.5 Pseudo-Elements — Styling Part of an Element

A pseudo-element (double colon ::) targets a specific PART of an element's content — such as just the first letter, or lets you insert generated content that doesn't exist in the HTML at all. Note: older CSS2 syntax used a single colon for these too (:before), but the modern standard is double-colon (::before).

Pseudo-Element	Targets / Inserts
::before	Inserts generated content immediately BEFORE the element's content
::after	Inserts generated content immediately AFTER the element's content
::first-letter	Styles only the FIRST letter of a text block
::first-line	Styles only the FIRST line of a text block
::selection	Styles text while the user is highlighting/selecting it
::placeholder	Styles the placeholder text inside an empty input field

```
/* ::before / ::after — the content property is REQUIRED, even if empty */
.required-field::after {
  content: " *";
  color: red;
}

.quote::before {
  content: "“";
  font-size: 2rem;
  color: teal;
}

/* Drop-cap effect — first letter larger */
p::first-letter {
  font-size: 2.5rem;
  font-weight: bold;
  color: navy;
}

/* Custom text-selection highlight color */
::selection {
```

```
  background: teal;
  color: white;
}

/* Style the placeholder text in a search box */
input::placeholder {
  color: #999;
  font-style: italic;
}
```

> ⚠️ **Pseudo-Class vs Pseudo-Element — Don't Mix Them Up:**
> Pseudo-CLASS (: single colon) → targets a STATE or POSITION of a real element → a:hover, li:first-child
> Pseudo-ELEMENT (:: double colon) → targets a PART of an element, or generates new content → p::first-letter, .box::before
> content: ""; is mandatory for ::before and ::after to render at all — forgetting it is the #1 beginner mistake.

4.6 Selector Specificity — Which Rule Wins?

When multiple CSS rules target the same element with conflicting styles, specificity decides which rule the browser applies. Specificity is calculated by counting how many of each selector type are used.

Selector Type	Specificity Weight	Example
Inline style	1000 (highest)	style="color:red;"
ID selector	100	#header
Class, attribute, pseudo-class	10	.box, [type="text"], :hover
Element, pseudo-element	1 (lowest)	p, ::before

```
p          { color: black; }   /* specificity = 1   */
.intro     { color: blue;  }   /* specificity = 10  */
#main p    { color: green; }   /* specificity = 101 */

/* #main p WINS — highest specificity, regardless of order in the file */
```

> 💡 **Golden Rules of Specificity:**
> Higher specificity always wins, no matter which rule is written first or last in the file.
> If specificity is EQUAL, the rule that appears LAST in the stylesheet wins.
> !important overrides everything (use very sparingly — it makes debugging harder).

5. Background Properties

CSS provides several properties to control the background of any element — solid colors, images, gradients, and more.

5.1 Core Background Properties

Property	Purpose	Example Value
background-color	Solid background color	red, #ff0000, rgba(0,0,0,0.5)
background-image	Sets an image as background	url('bg.jpg')
background-repeat	Controls image tiling	repeat, no-repeat, repeat-x, repeat-y
background-position	Positions the background image	center, top left, 20px 50px
background-size	Scales the background image	cover, contain, 100px 100px
background-attachment	Scroll behavior of the image	scroll, fixed, local

```
/* Individual properties */
.hero {
  background-color: #1e3a5f;
  background-image: url('banner.jpg');
  background-repeat: no-repeat;
  background-position: center;
  background-size: cover;
  background-attachment: fixed;
}

/* Shorthand — same result in one line */
.hero {
  background: #1e3a5f url('banner.jpg') no-repeat center/cover fixed;
}
```

5.2 Gradients as Backgrounds

```
/* Linear gradient — smooth color transition in a direction */
.box1 {
  background: linear-gradient(to right, teal, navy);
}

/* Angled gradient */
.box2 {
  background: linear-gradient(45deg, red, orange, yellow);
}

/* Radial gradient — circular from center outward */
.box3 {
  background: radial-gradient(circle, white, teal);
}
```

> 💡 **Practical Tips:**
> background-size: cover → fills the entire element, may crop the image slightly
> background-size: contain → fits the whole image inside, may leave empty space
> Use background-attachment: fixed for parallax scrolling hero sections
> Always test background images on mobile — large images can slow page load

6. Manipulating Text and Font

6.1 Text Properties

Property	Purpose	Example Value
color	Sets text color	navy, #333, rgb(50,50,50)
text-align	Horizontal alignment	left, center, right, justify
text-decoration	Underline/strikethrough/none	underline, line-through, none
text-transform	Changes letter casing	uppercase, lowercase, capitalize
text-indent	Indents first line of text	30px, 2em
letter-spacing	Space between letters	1px, 0.05em
line-height	Space between lines of text	1.5, 24px
text-shadow	Adds a shadow behind text	2px 2px 4px gray

```
p {
  color: #333333;
  text-align: justify;
  text-transform: capitalize;
  line-height: 1.6;
  letter-spacing: 0.5px;
}

h1 {
  text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
  text-decoration: underline;
}
```

6.2 Font Properties

Property	Purpose	Example Value
font-family	Sets the typeface	'Poppins', Arial, sans-serif
font-size	Sets text size	16px, 1.2rem, 120%
font-weight	Sets text boldness	normal, bold, 400, 700
font-style	Sets italic/oblique style	normal, italic, oblique
font-variant	Small caps styling	normal, small-caps

```
body {
  font-family: 'Poppins', Arial, sans-serif;
  font-size: 16px;
}

h1 {
  font-weight: 700;
  font-style: normal;
}
```

```
/* Shorthand: style weight size/line-height family */
p {
  font: italic 400 16px/1.5 'Roboto', sans-serif;
}
```

> 💡 **Using Google Fonts:**
>
> 1. Visit fonts.google.com and choose a font
> 2. Copy the provided <link> tag into your HTML <head>
> 3. Use it: font-family: 'Poppins', sans-serif;
>
> Always include a generic fallback (sans-serif, serif) in case the custom font fails to load.

7. The Box Model

Every single HTML element on a page is treated by CSS as a rectangular box. Understanding the Box Model is essential to controlling spacing and layout precisely.

7.1 The Four Layers (Outside → Inside)

Layer	Description
Margin	Transparent space OUTSIDE the border — separates this element from others
Border	Visible line that wraps around the padding and content
Padding	Space INSIDE the border, between the border and the actual content
Content	The actual text, image, or child elements inside the box

> **Visualize it like a framed photograph:**
> Content = The photo itself
> Padding = The white matting around the photo inside the frame
> Border = The physical picture frame
> Margin = The empty wall space around the hung frame

7.2 Box Model Properties

```
.box {
  width: 300px;
  height: 150px;

  padding: 20px;              /* all sides equal */
  padding: 10px 20px;         /* top/bottom  left/right */
  padding: 10px 20px 15px 5px;/* top right bottom left */

  border: 2px solid teal;     /* width style color */

  margin: 20px auto;          /* top/bottom = 20px, left/right = auto (centers
block) */
}
```

7.3 box-sizing — The Most Important Property

By default, width and height apply ONLY to the content — padding and border are added EXTRA on top, making elements bigger than expected. The box-sizing property fixes this.

```
/* DEFAULT behavior — confusing! */
.box-default {
  box-sizing: content-box;  /* default value */
  width: 300px;
  padding: 20px;
  border: 5px solid black;
```

```
  /* Actual rendered width = 300 + 40 + 10 = 350px !! */
}

/* RECOMMENDED behavior — predictable */
.box-fixed {
  box-sizing: border-box;
  width: 300px;
  padding: 20px;
  border: 5px solid black;
  /* Actual rendered width = exactly 300px */
}

/* Apply to the ENTIRE page — best practice */
* {
  box-sizing: border-box;
}
```

> ⭐ **Golden Rule for Every Project:**
>
> Always start your stylesheet with: * { box-sizing: border-box; }
>
> This makes width/height calculations predictable and prevents layout bugs.

8. Styling Lists

CSS provides dedicated properties to customize how bullet points and numbered list markers appear — essential for building clean navigation menus.

8.1 List Style Properties

Property	Purpose	Example Value
list-style-type	Changes the bullet/number style	disc, circle, square, decimal, none
list-style-image	Uses a custom image as the bullet	url('star.png')
list-style-position	Bullet inside or outside the text box	inside, outside
list-style	Shorthand for all three properties above	square inside

```
ul {
  list-style-type: square;
  list-style-position: inside;
}

ol {
  list-style-type: upper-roman;
}

/* Remove bullets completely — common for navigation menus */
.nav-list {
  list-style: none;
  margin: 0;
  padding: 0;
}
```

8.2 Building a Navigation Menu from a List

Most website navigation bars are simply a <ul> list styled with CSS to remove bullets and lay items out horizontally:

```
<ul class="nav-menu">
  <li><a href="#">Home</a></li>
  <li><a href="#">About</a></li>
  <li><a href="#">Courses</a></li>
  <li><a href="#">Contact</a></li>
</ul>

<style>
.nav-menu {
  list-style: none;    /* remove bullets */
  margin: 0;
  padding: 0;
  display: flex;        /* lay items out horizontally */
}

.nav-menu li {
```

```
  margin-right: 20px;
}

.nav-menu a {
  text-decoration: none;
  color: navy;
  font-weight: bold;
}

.nav-menu a:hover {
  color: teal;
}
</style>
```

> 💡 **Key Insight:**
> list-style: none + display: flex on the <ul> is the standard technique used to build almost every horizontal navigation bar on the web.

9. CSS Positioning

The position property controls exactly where an element appears on the page, and how it interacts with surrounding elements.

9.1 The Five Position Values

Value	Behavior
static	Default. Follows normal document flow — top/left/right/bottom have no effect
relative	Positioned relative to its own normal position. Still occupies its original space
absolute	Removed from normal flow; positioned relative to nearest positioned ancestor
fixed	Positioned relative to the browser window; stays in place while scrolling
sticky	Acts relative until a scroll threshold, then behaves like fixed

9.2 Code Examples

```
/* RELATIVE — nudges element from its normal spot, space is preserved */
.box-relative {
  position: relative;
  top: 10px;
  left: 20px;
}

/* ABSOLUTE — positioned relative to nearest ancestor with position set */
.parent {
  position: relative;   /* establishes positioning context */
}
.badge {
  position: absolute;
  top: 10px;
  right: 10px;
}

/* FIXED — stays in place even when page scrolls (e.g. floating buttons) */
.chat-button {
  position: fixed;
  bottom: 20px;
  right: 20px;
}

/* STICKY — sticks to the top once user scrolls past it */
.sticky-nav {
  position: sticky;
  top: 0;
}
```

9.3 z-index — Controlling Stack Order

When elements overlap, z-index determines which one appears on top. Higher values stack above lower values. It only works on elements that have a position value other than static.

```
.background-layer {
  position: relative;
  z-index: 1;
}

.foreground-layer {
  position: absolute;
  z-index: 10;   /* appears ABOVE the background-layer */
}
```

Common Use Case	Recommended Position Value
Sticky/fixed header navbar	sticky or fixed
Notification badge on an icon	absolute (with relative parent)
Floating WhatsApp/chat button	fixed
Slight nudge of an icon or text	relative
Normal page content	static (default — usually no need to set it)

Quick Reference — CSS Cheat Sheet

▸ Types of CSS

Type	Where It Lives
Inline	style="..." attribute directly on an HTML tag
Internal	<style> tag inside <head>
External	Separate .css file linked with <link>

▸ Basic & Combinator Selectors

Selector	Syntax
Element	p { }
Class	.name { }
ID	#name { }
Descendant (any depth)	div p { }
Child (direct only)	div > p { }
Adjacent sibling	h1 + p { }
General sibling	h1 ~ p { }

▸ Attribute Selectors

Syntax	Meaning
[attr]	Has the attribute
[attr="val"]	Exact match
[attr^="val"]	Starts with
[attr$="val"]	Ends with
[attr*="val"]	Contains anywhere

▸ Pseudo-Classes (single colon :)

Pseudo-Class	Meaning
:hover / :focus / :active	Mouse-over / focused / being clicked
:first-child / :last-child	First / last child of parent
:nth-child(odd/even/3n)	Positional pattern matching
:not(selector)	Everything except the given selector
:checked / :disabled	Checkbox state / form field state

▸ Pseudo-Elements (double colon ::)

Pseudo-Element	Meaning
::before / ::after	Insert generated content (needs content:"")
::first-letter / ::first-line	Style first letter / first line of text
::selection	Style user's text-highlight color
::placeholder	Style input placeholder text

▸ Specificity (low → high)

Rank	Selector Type
1 (lowest)	Element, pseudo-element — p, ::before
10	Class, attribute, pseudo-class — .box, :hover
100	ID selector — #header
1000 (highest)	Inline style — style="..."

▸ Background

Property	Purpose
background-color	Solid background color
background-image	Image as background
background-size: cover/contain	Scales background image
linear-gradient() / radial-gradient()	Gradient backgrounds

▸ Text & Font

Property	Purpose
color	Text color
font-family / font-size / font-weight	Typeface control
text-align / text-transform	Alignment & casing
line-height / letter-spacing	Spacing control

▸ Box Model

Property	Purpose
margin	Space outside the border
border	Line around the element

padding	Space inside the border
box-sizing: border-box	Predictable width/height calculation

▸ Lists

Property	Purpose
list-style-type	Bullet/number style
list-style: none	Removes bullets (used for nav menus)

▸ Positioning

Value	Behavior
static	Default flow (no effect from top/left)
relative	Nudged from normal position, space preserved
absolute	Positioned relative to nearest positioned ancestor
fixed	Stays fixed to the browser window
sticky	Relative until scroll point, then fixed
z-index	Controls stacking order of overlapping elements

Practice Questions — CSS

> **Instructions:**
> Attempt all questions. Short answer questions carry 2 marks each. Long answer / coding questions carry 5 marks each.

Section A — Short Answer Questions (2 marks each)

1. What is CSS? Why is it used alongside HTML?

2. Write the basic syntax of a CSS rule with an example.

3. Differentiate between Class selector and ID selector.

4. What is the difference between Internal and External CSS?

5. What is the purpose of the box-sizing property?

6. Name the four layers of the CSS Box Model in correct order.

7. What is the difference between position: relative and position: absolute?

8. What does the property list-style: none do, and where is it commonly used?

9. What is the difference between background-size: cover and contain?

10. What is z-index? On which position values does it work?

11. Differentiate between a pseudo-class and a pseudo-element, with one example of each.

12. What is the difference between the descendant selector (div p) and the child selector (div > p)?

13. What does the attribute selector a[href$=".pdf"] target? Give a real use case.

14. Why is the content property required in ::before and ::after?

15. What is CSS specificity? Which wins: #main p or .intro?

Section B — Long Answer Questions (5 marks each)

1. Explain the CSS Box Model in detail with a diagram description covering margin, border, padding, and content. (5 marks)

2. Explain the three types of CSS (Inline, Internal, External) with code examples and their order of priority. (5 marks)

3. Explain any six CSS selectors with syntax and example code for each. (5 marks)

4. Explain all five CSS position values (static, relative, absolute, fixed, sticky) with a code example for each. (5 marks)

5. What are Google Fonts? Explain the steps to add a custom font to a webpage using CSS. (5 marks)

6. Create a horizontal navigation bar using an unordered list styled with CSS (list-style, display, hover effects). (5 marks)

7. Explain the difference between structural pseudo-classes (:nth-child, :first-child) and interaction pseudo-classes (:hover, :focus) with two examples of each. Explain the nth-child(3n) and nth-child(odd) formulas. (5 marks)

8. Explain CSS specificity with a table of selector weights. Given three conflicting rules on the same element (element, class, ID selector), explain which one wins and why. (5 marks)

Section C — Coding Exercises

1. Write CSS to style a with: 300px width, 20px padding, 2px solid teal border, 20px margin, and box-sizing: border-box. (3 marks)

2. Create a hero banner section with a background image, background-size: cover, and centered white text using text-shadow. (5 marks)

3. Design a sticky navigation bar that stays fixed at the top of the page when scrolling, with a semi-transparent background using rgba(). (5 marks)

4. Using only CSS (no extra HTML classes), zebra-stripe a table's rows with :nth-child(odd/even), and add a red asterisk after every <label> using ::after to mark required fields. (5 marks)

5. Write an attribute selector rule that styles all external links (href starting with "https") with a green color and all PDF links (href ending with ".pdf") with a small icon added via ::after. (4 marks)